



智能移动技术 实验报告

课程名称 智能移动技术

成 员 xx

日 期 2025 年 3 月 24 日

指导老师 王越、熊蓉

一、方法介绍

在本次导航规划作业中,我们小组采用了 **RRT*** 进行全局路径规划,并结合 **DWA** 进行局部轨迹优化,以实现高效的路径规划与避障控制。

1.1 全局路径规划 - RRT*

RRT* 是一种基于随机采样的路径规划算法,具有全局收敛性和路径优化能力。其核心步骤如下:

1. **随机采样**: 在搜索空间内随机生成采样点,部分概率直接采样目标点以提高收敛速度。
2. **最近邻搜索**: 找到当前树中距离采样点最近的节点。
3. **步进扩展**: 按照一定步长沿最近邻方向生成新节点,并检测其可行性。
4. **邻居重连**: 在新节点周围搜索 k 个最近邻,并选择总代价最小的节点作为父节点。
5. **路径优化**: 尝试重新连接邻居节点以进一步减少路径长度。

1.2 局部轨迹优化 - DWA

DWA 主要用于局部避障和速度优化,确保机器人沿着全局路径平稳移动。其核心思想如下:

1. **动态窗口计算**: 根据当前速度、加速度约束,计算可行的速度范围。
2. **轨迹模拟**: 在速度空间内模拟未来轨迹,预测可能到达的位置。
3. **代价函数评估**: 综合考虑目标距离、避障安全性和速度平稳性,对每条轨迹进行打分:

$$J = w_1 f_{goal} + w_2 f_{obstacle} + w_3 f_{velocity} \quad (1)$$

其中:

- f_{goal} : 到目标点的距离代价
 - $f_{obstacle}$: 与障碍物的最小距离代价
 - $f_{velocity}$: 速度代价
4. **最优轨迹选择**: 选取代价函数值最高的速度对 (v, ω) 作为最终执行速度。

1.3 主程序流程

1. 采用 RRT* 进行全局路径规划,获取路径点序列。
2. 使用 DWA 在路径点之间进行局部避障与速度优化。
3. 结合传感器数据实时更新路径,如遇停滞或障碍物阻挡,触发重新规划。

该方法综合了 RRT* 的全局路径优化能力与 DWA 的实时避障能力,实现了高效的轨迹规划与运动控制。

二、实验结果

测试评分：根据时间，机器人需在两点之间往返到点，到点评判标准为到达给定点中心 10cm 半径范围内，往返次数 5 次，根据所用时间评分，时间少得分高；且计算碰撞次数，碰撞少得分高

最后的结果

1. Simple 模式用时 **41s**，碰撞次数 **0 次**
2. Hard 模式用时 **46s**，碰撞次数 **0 次**

三、分析讨论

3.1 路径规划算法选择及优化

在路径规划算法的选择上，我们主要考虑了环境复杂性、计算效率和路径质量。一般而言，常见的路径规划方法包括 A* 算法、人工势场法、RRT 及其改进版本等。但是考虑到，A* 算法适用于静态环境，在动态环境中计算代价较高；人工势场法容易陷入局部最优；RRT 具有较强的探索性，但路径往往较为曲折。而采用 RRT* 进行路径规划，能够在保证路径可行性的同时进行路径优化，生成更平滑、更短的路径。因此，在本次实验中，我们小组采用了 **RRT*** 做路径规划

针对 RRT* 算法的优化，我们主要聚焦于采样点的改进：

- 调整范围内的聚类采样点数量，以在搜索效果与计算时间之间取得平衡。
- 通过调整每次采样点的步长 (`step_size`)，避免在规划过程中采样点与障碍物相交，同时确保最终点能够到达指定范围。
- 通过优化采样策略 (`find_k_nearest_nodes` 函数)，使得采样点的生成趋向终点，加快路径收敛，提高采样效率。

Listing 1: 优化的采样策略

```
1 def select_sample_point(self, vision, goal_x, goal_y):
2     return (goal_x, goal_y) if random.random() < self.
        goal_sample_prob else self.generate_random_point(
            vision)
```

3.2 局部避障算法选择及优化

局部避障算法需要能够实时调整机器人的轨迹，以应对动态障碍物的变化。常见的方法包括人工势场法、向量势直方图法 (VFH) 和动态窗口法 (DWA)。人工势场法虽然计算简单，但容易陷入局部最优且存在震荡问题；VFH 通过构建概率直方图评估可行路径，但计算复杂度较高。因此，选择 DWA 作为局部避障方法，其核心优势在于直接在速度空间中优化轨迹，并考虑了机器人动力学约束，从而在避障的同时保持运动的平滑性和稳定性。

针对 DWA 算法的优化，我们进行了以下改进：

- 我们设计的 DWA 评估函数如下：

$$J = w_{goal} * dist_score + w_{obstacle} * velocity_score + w_{velocity} * goal_score \quad (2)$$

通过调整评估函数权重（位于 `evaluate_trajectory`），使得机器人在不同场景下能够自适应地在速度、避障和目标到达性之间做出平衡。

- 采用更精细的动态窗口划分（位于 `find_optimal_motion`），提高搜索精度，使轨迹更为平滑。
- 增强预测机制（位于 `update_sensor_data`），提高对动态障碍物的感知能力，使得避障更为灵活。具体优化如下：
 - 结合历史位置信息，计算障碍物的速度和方向，以预测其未来轨迹。
 - 在障碍物预测路径上进行安全性分析，提前调整运动策略，减少突发碰撞风险。
 - 采用多步预测方法，使机器人在复杂动态环境中更稳定地进行避障决策。

预测后面五步的代码如下：

```
1 for i in range(5): # 预测未来 5 个时间步的位置
2     predicted_pos = [robot.x, robot.y + np.sign(vel_y) * 50 * (i -
3         1), robot.orientation]
4     self.dynamic_obstacles.append(predicted_pos)
```

四、优化改进

4.1 实时路径规划

在仿真平台运行代码时，我们发现，在 `hard` 模式下，由于障碍物的移动，机器人有时会偏离 RRT* 规划的路径，甚至停滞不前。我们猜测是因为 DWA 计算出的各个方向的最优轨迹可能不足以让机器人回到规划路径，导致其在局部区域内震荡或陷入局部最优。在反复调节 DWA 参数后，我们仍然无法从根源上解决机器人过于偏离预定路径的现象。为了确保机器人能够在动态环境中自适应调整路径，我们在 DWA 轨迹规划中加入了 `if_stop` 函数，以实时判断机器人是否需要停止或重新规划路径。该函数实现如下：

Listing 2: 实时路径规划判断函数

```
1 def if_stop(self, distance_threshold=1000.0):
2     if np.linalg.norm(self.robot_speed) < 1e-4:
3         return True
4
5     if np.linalg.norm(self.robot_state[:2] - self.goal_pos) >
6         distance_threshold:
7         print(f"\nExceeds threshold {distance_threshold}, stopping.")
```

```

7         return True
8
9         return False

```

该函数主要作用如下：

- 当机器人速度接近零 ($< 10^{-4}$) 时，认为其进入停滞状态，从而触发路径重新规划。
- 如果机器人当前位置与目标点之间的距离超过设定的阈值 (*distance_threshold*)，则认为机器人偏离了规划范围过远，需要停止并重新规划路径。

4.2 终点转向规划设计

在原始方案中，我们仅通过一个简单的判断函数检测机器人是否到达终点，之后就针对新的目标点进行路径规划了。

但在仿真过程中，我们发现，这种方式存在明显的问题：由于未对终点处的机器人姿态进行控制，机器人在重新规划路径后，其初始朝向可能未对准新的目标方向，且由于速度过快，从而需要绕大圈才能回到正确轨迹上（如图 1），在这期间发生碰撞的概率会大幅度提高。

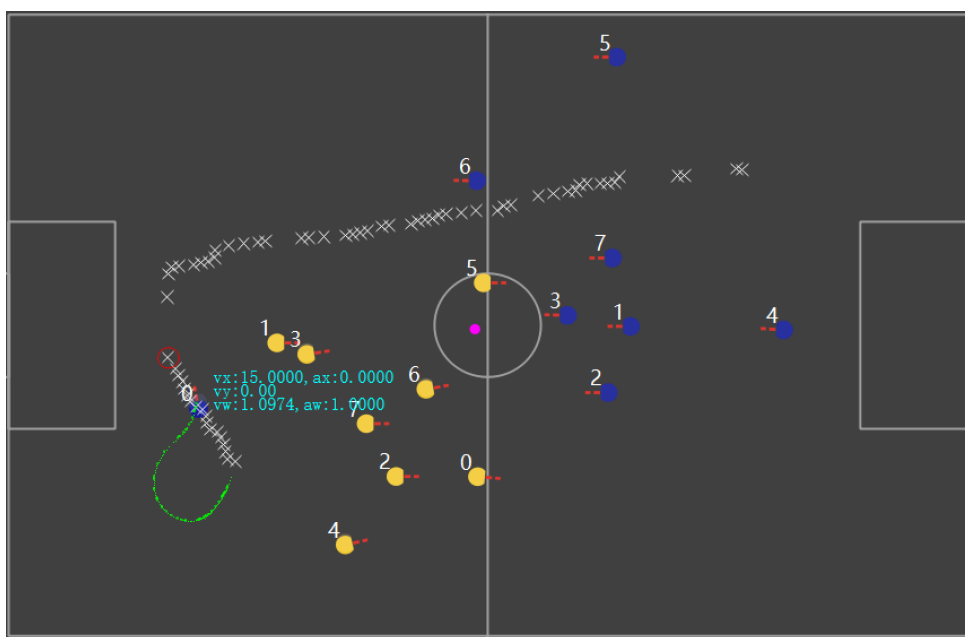


图 1 终点转向未规划状态下的机器人轨迹

为了解决这一问题，我们设计了一种终点转向策略，使机器人在到达终点后主动调整朝向，以更合理、更美观的方式衔接下一段路径。具体实现如下：

Listing 3: 终点转向控制代码

```

1 start_x, start_y = vision.my_robot.x, vision.my_robot.y
2 theta = math.atan2((goal_y - start_y), (goal_x - start_x))
3 origen = vision.my_robot.orientation

```

```

4
5 w_error = abs(origen - theta)
6 if w_error > math.pi:
7     w2 = -3.0
8 else:
9     w2 = 3.0
10
11 # 发送旋转指令，使机器人对准新目标方向
12 action.sendCommand(vx=0, vy=0, vw=w2)
13 while abs(origen - theta) > 0.2:
14     origen = vision.my_robot.orientation
15
16 # 终止旋转，确保机器人稳定
17 action.sendCommand(vx=0, vy=0, vw=0)
18 time.sleep(0.1)
19 flag = flag + 1

```

该方法的核心思想如下：

- 计算机器人当前位置与新目标点之间的角度偏差 θ 。
- 判断当前机器人朝向 *origen* 是否已对准 θ ，若未对准，则执行旋转指令。
- 通过 w_{error} 控制旋转方向，确保机器人以最优方式调整姿态。
- 旋转完成后，停止角速度调整，并在短暂延时后进入下一段路径规划。

该优化方案有效减少了机器人在终点处的无规划状态，使其能够快速衔接下一段任务路径，防止碰撞，避免不必要的绕行，同时提高了整体轨迹的平滑度和美观性。

附录 A 文件列表

文件名	功能描述
code	实验代码
智能移动技术实验报告-16.pdf	实验报告
README.md	文件简介
LOG	实验结果

附录 B 人员分工

成员	任务分工	贡献度
	代码编写和报告撰写	33.33%
	代码编写和报告撰写	33.33%
	代码编写和报告撰写	33.33%